

Ćwiczenie laboratoryjne CA

Automaty Wolframa – jednowymiarowe automaty komórkowe (ECA)

1. Cel ćwiczenia

Celem ćwiczenia jest zapoznanie studentów z jednowymiarowymi automatami komórkowymi (ECA – Elementary Cellular Automata) zaproponowanymi przez Stephena Wolframa.

Po wykonaniu ćwiczenia student powinien:

- rozumieć ideę reguł Wolframa (0–255),
- potrafić zaimplementować automat jednowymiarowy,
- wizualizować jego działanie w postaci diagramu czasoprzestrzennego,
- analizować zachowanie różnych reguł automatu.

2. Wprowadzenie teoretyczne

Jednowymiarowy automat komórkowy składa się z jednego rzędu komórek.

Każda komórka może przyjmować dwa stany:

0 – martwa

1 – aktywna

Stan komórki w kolejnym kroku czasu zależy od trzech komórek: lewej, centralnej i prawej.

Możliwych jest 8 konfiguracji sąsiedztwa:

111, 110, 101, 100, 011, 010, 001, 000.

Dla każdej konfiguracji definiowany jest wynik (0 lub 1).

Zestaw ośmiu wyników tworzy regułę automatu.

3. Numerowanie reguł

Reguła automatu zapisywana jest jako liczba od 0 do 255.

Przykład:

Reguła 30 ma zapis binarny:

00011110

Oznacza to, że dla kolejnych konfiguracji sąsiedztwa przypisane są odpowiednie stany komórki w następnym kroku czasu.

Zadanie 1 – implementacja automatu

Zaimplementuj automat Wolframa w języku Python.

Program powinien:

- tworzyć jednowymiarową tablicę komórek o długości N (np. 101),
- umożliwiać wybór numeru reguły (0–255),
- wykonywać symulację przez określoną liczbę iteracji,
- zapisywać kolejne stany automatu,
- wyświetlać wynik w formie diagramu czasoprzestrzennego.

Zadanie 2 – analiza wybranych reguł

Uruchom symulację dla następujących reguł:

- 30
- 90
- 110

Dla każdej reguły przeprowadź symulację dla dwóch warunków początkowych:

- A) jedna aktywna komórka w środku
- B) losowy rozkład komórek

W sprawozdaniu umieść diagramy dla wszystkich przypadków.

Zadanie 3 – analiza statystyczna

Dla każdej reguły oblicz:

1. Średnią gęstość aktywnych komórek w czasie.
2. Zmienność układu – ile komórek zmienia stan między kolejnymi krokami.
3. Przedstaw wykres entropii w czasie dla reguły 30, 90, 110.

Porównaj wyniki dla analizowanych reguł.

Zadanie 4 – klasy Wolframa

Na podstawie uzyskanych diagramów spróbuj przypisać każdą regułę do jednej z czterech klas zachowania:

Klasa I – stabilizacja

Klasa II – struktury okresowe

Klasa III – chaos

Klasa IV – złożone struktury

Uzasadnij swój wybór.

Zadanie rozszerzone (opcjonalne)

Przeprowadź eksperyment polegający na losowym przeszukiwaniu przestrzeni wszystkich reguł (0–255).

Znajdź reguły, które generują najbardziej złożone wzory.

9. Wymagania do sprawozdania

Sprawozdanie powinno zawierać:

- opis działania automatu,
- opis implementacji programu,
- diagramy czasoprzestrzenne dla wybranych reguł,
- analizę wyników,
- wnioski.

Szkielet rozwiązania

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# KONFIGURACJA (studenci mogą zmieniać)
# =====
RULE = 30          # 0..255 (np. 30, 90, 110)
N = 101           # liczba komórek
STEPS = 150        # liczba kroków czasu
INIT_MODE = "single" # "single" albo "random"
P_RANDOM = 0.5     # gęstość jedynek dla INIT_MODE="random"
BOUNDARY = "wrap"  # "wrap" (periodyczne) albo "fixed" (zera na brzegach)
SEED = 0          # ziarno losowości

# =====
# 1) KONWERSJA REGUŁY NA BITY
# =====
def rule_to_bits(rule: int) -> np.ndarray:
    """
    Zamienia regułę 0..255 na tablicę 8 bitów.
    Kolejność powinna odpowiadać konfiguracjom:
    111, 110, 101, 100, 011, 010, 001, 000
    """
    # TODO 1:
    # - sprawdź zakres rule
    # - zamień rule na binarny zapis 8-bit (np.binary_repr)
    # - zwróć jako np.ndarray dtype=int (lub uint8)

    raise NotImplementedError("TODO: uzupełnij rule_to_bits()")

# =====
# 2) INICJALIZACJA STANU POCZĄTKOWEGO
# =====
def init_cells(n: int, mode: str = "single", p: float = 0.5, seed: int
```

```

= 0) -> np.ndarray:
    """
    Tworzy stan początkowy (wektor długości n).
    mode:
        - 'single': jedna jedynka w środku
        - 'random': losowy wektor 0/1 o gęstości p
    """

    # TODO 2:
    # - utwórz rng = np.random.default_rng(seed)
    # - utwórz cells = np.zeros(n)
    # - jeśli mode == 'single' ustaw środek na 1
    # - jeśli mode == 'random' wylosuj 0/1 z prawdopodobieństwem p
    # - zwróć np.ndarray dtype=np.uint8

    raise NotImplementedError("TODO: uzupełnij init_cells()")

# =====
# 3) JEDEN KROK AUTOMATU (ECA)
# =====
def step_eca(cells: np.ndarray, rule_bits: np.ndarray, boundary: str =
"wrap") -> np.ndarray:
    """
    Wykonuje jeden krok automatu Wolframa.
    boundary:
        - 'wrap': periodyczne (torus)
        - 'fixed': stałe zera poza brzegami
    """
    n = len(cells)
    new = np.zeros(n, dtype=np.uint8)

    # TODO 3a: napisz funkcję get(i) obsługującą brzeg
    # - 'wrap': zwróć cells[i % n]
    # - 'fixed': jeśli i poza [0, n-1] zwróć 0, inaczej cells[i]
    def get(i: int) -> int:
        raise NotImplementedError("TODO: uzupełnij get()")

    # TODO 3b: dla każdej komórki i:
    # - odczytaj left, center, right
    # - policz pattern = (left<<2) | (center<<1) | right (0..7)
    # - odczytaj wynik z rule_bits
    # UWAGA: rule_bits jest w kolejności [111..000], więc indeks = 7
    - pattern
    # - ustaw new[i]
    for i in range(n):
        raise NotImplementedError("TODO: uzupełnij pętlę w step_eca()")

    return new

```

```

# =====
# 4) SYMULACJA
# =====
def simulate(rule: int, n: int, steps: int, init_mode: str, p_random:
float,
            boundary: str, seed: int) -> np.ndarray:
    """
    Zwraca historię: macierz (steps+1) x n
    """
    # TODO 4:
    # - rule_bits = rule_to_bits(rule)
    # - cells = init_cells(...)
    # - utwórz history = np.zeros((steps+1, n), dtype=np.uint8)
    # - history[0] = cells
    # - pętla po czasie: cells = step_eca(...), history[t] = cells
    # - zwróć history

    raise NotImplementedError("TODO: uzupełnij simulate()")

# =====
# 5) METRYKI: gęstość i zmienność
# =====
def metrics(history: np.ndarray) -> tuple[float, float]:
    """
    Zwraca:
    - rho: średnia gęstość jedynek
    - V: średnia zmienność (odsetek komórek zmieniających stan między
    krokami)
    """
    # TODO 5:
    # - rho = history.mean()
    # - diffs = (history[1:] != history[:-1]).mean(axis=1)
    # - V = diffs.mean()
    # - zwróć (rho, V)

    raise NotImplementedError("TODO: uzupełnij metrics()")

# =====
# 6) WIZUALIZACJA
# =====
def plot_history(history: np.ndarray, rule: int, init_mode: str,
boundary: str):
    plt.figure(figsize=(10, 6))
    plt.imshow(history, cmap="binary", interpolation="nearest",
aspect="auto")
    plt.title(f"Automat Wolframa - reguła {rule} | init={init_mode} |

```

```

brzeg={boundary}")
    plt.xlabel("Komórki")
    plt.ylabel("Czas")
    plt.show()

# =====
# MAIN
# =====
if __name__ == "__main__":
    # TODO 6 (opcjonalnie): dodaj testy porównawcze dla reguł 30/90/110
    # i dla init_mode: 'single' oraz 'random'
    history = simulate(
        rule=RULE, n=N, steps=STEPS,
        init_mode=INIT_MODE, p_random=P_RANDOM,
        boundary=BOUNDARY, seed=SEED
    )

    rho, V = metrics(history)
    print(f"Reguła: {RULE}")
    print(f"Średnia gęstość (rho): {rho:.4f}")
    print(f"Zmienność (V): {V:.4f}")

    plot_history(history, RULE, INIT_MODE, BOUNDARY)

```